



ONT401 Semester 1 – Assignment 1

Due Date 22 May 2026 by 23:59

Learning Outcomes

Upon successful completion of this assignment, you should be able to:

- Apply software design patterns in a practical business scenario
- Design maintainable and modular OOP systems
- Justify software architecture decisions
- Produce UML documentation
- Collaborate effectively in teams

Assignment Rules

- You are required to work in groups of 3 to 5 people.
- Only 1 group member should submit.
- All group members' details should be provided in the assignment.
- **Late** assignment submissions will **lose 25% per day**.

Each group is required to do **only 1 of the 3 following assignments**

(Assignment A OR B OR C)

For each assignment, come up with your own project title and use a tech stack of your choice.

Please note page 14

Assignment A: Combat Game with Design Patterns (C#)

- You can implement this using Unity (recommended), Windows Forms, or Console Application.
- If using Unity, include a .unitypackage for easy import.

1. Assignment Overview

Design and develop a **Game System** that allows players to create characters, choose game worlds/factions, customize abilities/equipment, receive in-game event notifications, and manage gameplay interactions.

Your system must demonstrate practical implementation of the following software design patterns:

1. Strategy
2. Observer
3. Decorator
4. Factory Method
5. Abstract Factory
6. Singleton

2. Scenario

A game development studio wants to build the prototype of a role-playing/adventure game where players can create characters, select classes/factions, equip items, and receive real-time game event updates.

The platform must support:

- Multiple character types
- Different factions/world themes
- Dynamic equipment/ability upgrades
- Flexible combat/attack strategies
- Event/notification systems
- Centralized game management/configuration

Your development team has been contracted to build the first prototype.

3. Functional Requirements

3.1 Character Creation

Players must be able to create different character types:

- Warrior
- Mage
- Archer
- Assassin
- Tank

Use: **Factory Method Pattern**

3.2 Faction / World Families

The game must support different world/faction themes:

- Medieval Kingdom
- Sci-Fi Universe
- Fantasy Realm

Each faction/world should produce related game assets.

Examples:

- Character Skins
- Weapons
- Mounts/Pets
- Environment Themes

Use: **Abstract Factory Pattern**

3.3 Character Customization

Players must be able to dynamically equip/upgrade characters.

Examples:

- Armor Upgrade
- Magic Buff
- Speed Boost
- Extra Health
- Fire Damage Enchantment

Use: **Decorator Pattern**

3.4 Combat / Ability Strategy

The system must support interchangeable gameplay/combat strategies.

Examples:

- Melee Attack Strategy
- Ranged Attack Strategy
- Magic Attack Strategy
- Defensive Strategy
- Stealth Strategy

Use: **Strategy Pattern**

3.5 In-Game Notifications / Events

When important game events occur, multiple components must be notified.

Examples:

- Player Heads-Up Display Updated
- Quest Log Updated
- Enemy AI Triggered
- Achievement System Updated

Use: **Observer Pattern**

3.6 Shared Central Resource

Implement a globally shared singleton resource.

Examples:

- Game Engine Manager
- Configuration Manager
- Save/Load Manager
- Event Logger

Use: **Singleton Pattern**

4. Example Workflow Demonstration

1. Player selects **Fantasy Realm**
2. Factory creates **Mage Character**
3. Player equips **Magic Buff & Fire Enchantment**
4. Combat Strategy set to **Magic Attack**
5. Singleton Game Manager records state
6. Enemy AI / HUD / Quest Log notified of battle event
7. Achievement unlocked and displayed

5. Bonus Challenges (Optional)

For Additional Marks:

- Save/Load Functionality
- Inventory System
- Multiplayer Simulation
- Leveling / XP Mechanics

Assignment B: Laundry Pickup & Delivery App using Design Patterns

1. Assignment Overview

Design and develop a **Laundry Pickup & Delivery Application** that allows customers to schedule laundry pickups, select laundry services, customize orders, make payments, and receive order updates.

Your system must demonstrate practical implementation of the following software design patterns:

1. Strategy
2. Observer
3. Decorator
4. Factory Method
5. Abstract Factory
6. Singleton

2. Scenario

A startup laundry service wants to digitize its operations through a mobile/web platform where customers can:

- Book laundry pickups
- Select service types
- Customize washing preferences
- Pay online
- Track order progress
- Receive notifications

Your development team has been contracted to build the first prototype.

3. Functional Requirements

3.1 Laundry Service Creation

Users must be able to select different laundry service types:

- Wash Only
- Wash and Fold
- Dry Cleaning
- Ironing Service
- Express Same-Day Service

Use: **Factory Method Pattern**

3.2 Laundry Package / Provider Families

The system must support different service package tiers/providers:

- Standard Laundry Package
- Premium Laundry Package
- Business/Corporate Laundry Package

Each package should create related service families.

Examples:

- Packaging Type
- Delivery Method
- Detergent Type / Fabric Care Level

Use: **Abstract Factory Pattern**

3.3 Order Customization

Customers must be able to dynamically add optional extras.

Examples:

- Fabric Softener
- Stain Removal
- Perfume Treatment

- Delicate Wash
- Extra Folding
- Hanger Packaging

Use: **Decorator Pattern**

3.4 Payment / Pricing Strategy

System must support interchangeable pricing/payment strategies.

Examples:

- Credit Card
- EFT
- PayFast
- Subscription Discount
- Promo Code Discount
- Bulk Weight Pricing

Use: **Strategy Pattern**

3.5 Order Status Notifications

When laundry order status changes, multiple stakeholders must be notified.

Examples:

- Customer notified when pickup is assigned
- Driver notified of new collection
- Laundry Staff notified of incoming order
- Customer notified when order is out for delivery

Use: **Observer Pattern**

3.6 Shared Central Resource

Implement a globally shared singleton resource.

Examples:

- Order Logger
- Database Connection
- Configuration Manager
- Delivery Route Manager

Use: **Singleton Pattern**

4. Example Workflow Demonstration

1. Customer selects **Premium Laundry Package**
2. Factory creates **Dry Cleaning Service**
3. Customer adds **Stain Removal and Perfume Treatment**
4. Payment Strategy selected
5. Singleton Logger records booking
6. Pickup Driver notified
7. Customer receives status updates throughout process

5. Bonus Challenges (Optional)

For additional marks:

- Driver Route Optimization
- SMS/Email Notification Simulation
- Loyalty Rewards System

Assignment C: Food Ordering & Delivery App Using Design Patterns

1. Assignment Overview

Design and develop a **Food Ordering and Delivery Application** that allows customers to browse restaurants, order food, customize meals, pay online, and track delivery progress.

Your system must demonstrate practical implementation of the following software design patterns:

1. Strategy
2. Observer
3. Decorator
4. Factory Method
5. Abstract Factory
6. Singleton

2. Scenario

A startup food delivery company wants to build a digital platform similar to Uber Eats/Mr D that allows users to order food from restaurants and have it delivered.

The platform must support:

- Different meal categories
- Multiple restaurant/brand offerings
- Meal customization
- Flexible payment/pricing options
- Real-time order notifications
- Centralized order/configuration management

Your development team has been contracted to build the first prototype.

3. Functional Requirements

3.1 Meal Creation

Users must be able to order different meal/product types:

- Burger Meal
- Pizza
- Pasta
- Drinks
- Dessert

Use: **Factory Method Pattern**

3.2 Restaurant / Brand Families

The system must support different restaurant brands/categories:

- Fast Food Restaurant
- Premium Restaurant
- Healthy Food Restaurant

Each restaurant family should produce related offerings.

Examples:

- Menu Item
- Packaging Type
- Delivery Packaging / Branding

Use: **Abstract Factory Pattern**

3.3 Meal Customization

Users must be able to dynamically customize meals.

Examples:

- Extra Cheese
- Bacon

- Double Patty
- Large Portion Upgrade
- Extra Sauce
- Gluten-Free Base

Use: **Decorator Pattern**

3.4 Payment / Pricing Strategy

System must support interchangeable payment/pricing methods.

Examples:

- Credit Card
- EFT
- PayFast
- Promo Code Discount
- Loyalty Discount
- Surge Delivery Fee Strategy

Use: **Strategy Pattern**

3.5 Order Status Notifications

When order status changes, multiple parties must be notified.

Examples:

- Customer receives order confirmation
- Restaurant notified of new order
- Driver notified of pickup
- Customer notified when order is out for delivery

Use: **Observer Pattern**

3.6 Shared Central Resource

Implement a globally shared singleton resource.

Examples:

- Order Logger
- Database Connection
- App Configuration Manager
- Delivery Queue Manager

Use: **Singleton Pattern**

4. Example Workflow Demonstration

1. Customer selects **Healthy Food Restaurant**
2. Factory creates **Burger Meal**
3. Customer adds **Extra Cheese and Bacon**
4. Payment Strategy selected
5. Singleton Logger records order
6. Restaurant and Driver notified
7. Customer receives live order updates

5. Bonus Challenges (Optional)

For Additional Marks:

- Delivery Driver Tracking Simulation
- Database Persistence
- Loyalty Rewards System
- Multi-Restaurant Cart Support

The following apply to ALL 3 Assignments

6. Non-Functional Requirements

Your application must:

- Follow OOP Principles
- Demonstrate Loose Coupling
- Be Extendable
- Compile and Run Successfully
- Include Code Documentation

7. Deliverables

7.1 Source Code

Submit:

- Full Source Code
- Supporting Files
- Proper Folder Structure
- ReadMe with Run Instructions

7.2 Documentation Report (No more than 5 pages)

Must include:

Section A: Problem Analysis

- Scenario Understanding
- Assumptions Made

Section B: Design Decisions

- Justification of Pattern Usage
- Architecture Explanation

Section C: UML Diagrams

- Full Class Diagram
- Use Case Diagram

Section D: Reflection

- Team Collaboration Reflection
- Challenges Encountered
- Lessons Learned

7.3 Presentation and Demo

Each group to **present and demo** within **15 minutes** showing:

- Working Application
- Pattern Usage
- Runtime Behavior
- Group Member Contributions

8. Assessment Rubric for ALL 3 (100 Marks)

Criterion	Marks
Strategy Pattern	10
Observer Pattern	10
Decorator Pattern	10
Factory Method Pattern	10
Abstract Factory Pattern	10
Presentation and Documentation	30
UI and UX	5
Creativity	5
Code Quality / OOP Principles	10

End of Assignment